

NexaClip

AI Creator Intelligence Platform

Complete Web Application UX Flow

End-to-End User Journey & Interaction Design · Desktop Web App
Version 1.0 · March 2026 · For UI/UX Designer & Solution Architect

Actor Color Key	State Color Key
Blue = User action on screen	Loading = data or AI processing
Teal = System / backend response	Empty = no content yet exists
Purple = Claude AI involvement	Error = something went wrong
Red = Error condition	Success = action completed

Document Scope & Purpose

This document defines the complete end-to-end UX flow for the NexaClip web application.

It is structured as a sequence of user journeys — from first visit through every feature interaction. Each journey shows: what the user sees on their desktop browser, what happens in the backend, what state the UI enters, and how errors are handled.

This is a WEB APPLICATION document. All layouts are desktop-first (1280px+).

There is no mobile app scope in this document.

SECTIONS COVERED:

- 1. Site Map & Navigation Architecture
- 2. Journey 1 — First Visit & Registration
- 3. Journey 2 — Claude Onboarding Chat
- 4. Journey 3 — Image & Meme Creation
- 5. Journey 4 — Gaming Clip Upload & Editing
- 6. Journey 5 — Social Feed & Engagement
- 7. Journey 6 — User Profile Management
- 8. Journey 7 — Analytics Dashboard & Weekly Report
- 9. Journey 8 — Creator Coach (Claude Chat)
- 10. Journey 9 — Free-to-Pro Upgrade
- 11. Journey 10 — Settings & Account Management

Document Scope & Purpose
12. Cross-Journey System Flows (moderation, notifications)
13. Error Handling & Recovery Flows
14. Web-Specific UX Patterns Reference

1. Site Map & Navigation Architecture

The NexaClip web application has two distinct zones: the public marketing zone (accessible before login) and the authenticated product zone (accessible after login). Navigation, layout, and visual language differ between zones.

1.1 Complete Site Map

Zone	URL Path	Page Name	Auth Required	Primary Purpose
Public	nexaclip.com	Homepage (W01)	No	Convert visitors to signups
Public	/features	Features (W02)	No	Deep-dive product demo
Public	/pricing	Pricing (W03)	No	Plan comparison and signup
Public	/login	Login (W04)	No (redirects to /feed if authed)	Return user authentication
Public	/signup	Signup (W05)	No (redirects to /onboarding if authed)	New user registration
Public	/blog	Blog (W06)	No	Content marketing
Public	/p/:id	Public Post View	No	Shared content preview
Auth	/onboarding	Onboarding Chat	Yes (new users only)	Claude setup — shown once
Auth	/feed	Home Feed (A01)	Yes	Daily content consumption
Auth	/create	Create Hub (A02)	Yes	Content creation gateway
Auth	/create/image	Image Studio (A03)	Yes	AI image & meme generation
Auth	/create/clip	Clip Upload (A04a)	Yes	Video upload entry
Auth	/create/clip/:id/edit	Clip Editor (A04b)	Yes	Trim + AI polish + publish
Auth	/analytics	Analytics (A05)	Yes	Performance dashboard
Auth	/coach	Creator Coach (A06)	Yes (Pro: unlimited, Free: 3 msgs)	Claude AI chat coaching
Auth	/feed/post/:id	Post Detail (A07)	Yes	Full post with comments
Auth	/users/:id	User Profile (A08)	Yes	Other creator profile
Auth	/profile	Own Profile (A09)	Yes	Self profile view

Zone	URL Path	Page Name	Auth Required	Primary Purpose
Auth	/profile/edit	Edit Profile (A10)	Yes	Update profile data
Auth	/my-content	Content Library (A11)	Yes	All created content management
Auth	/upgrade	Upgrade Page (A12)	Yes	Subscription upgrade
Auth	/settings	Settings (A13)	Yes	Account, billing, preferences
System	/404	Not Found	No	Error recovery

1.2 Navigation Architecture by Zone

Navigation Component	Marketing Zone	App Zone
Primary nav	TopNav — fixed 72px. Logo + 4 nav links + 2 CTA buttons. Transparent on hero, white + shadow after scroll.	SideNav — fixed left 240px. Collapsible to 64px icon-only. All app navigation lives here.
Secondary nav	None — all links in TopNav	TopBar — 64px fixed top. Page title + search + notifications + user menu.
Page transitions	Full page loads (standard links)	SPA client-side routing (Angular Router). Fade transition 200ms between routes.
Auth state check	Every page: if JWT valid and unexpired → redirect to /feed	Every page: if no valid JWT → store current URL in localStorage, redirect to /login
Footer	4-column footer on all marketing pages	No footer in app zone — sidebar is the persistent navigation
Breadcrumbs	None — flat marketing structure	Present on deep pages: Create → Clip Editor. Analytics → Weekly Report.

1.3 URL History & Browser Navigation

Browser Behaviour	Implementation
Back button support	Angular Router uses HTML5 History API. Back button navigates within SPA correctly. No page reloads inside the app zone.
URL sharing	All /feed/post/:id URLs are publicly accessible with Open Graph metadata for social sharing previews. Auth-only pages redirect to login then return after auth.
Deep link handling	Push notifications link directly to specific screens. Unauthenticated deep links stored in localStorage as nx_return_to. Restored after login.
Tab title	Dynamic Angular Title service. Format: Page Name — NexaClip. E.g. "Analytics — NexaClip", "Home Feed — NexaClip".
Favicon	32px purple NexaClip logomark. No notification badge on favicon for MVP.

Journey 1 — First Visit & Registration

First impression through account creation · 3–5 minutes

This journey covers a new visitor arriving at nexaclip.com for the first time through to completing account creation and entering the onboarding flow. The primary goal of this journey is to convert intent into a registered account with minimum friction.

J1.1 — First Visit Journey Map

Awareness (Before site)	User Steps	Emotional State	Web UX Consideration
	User encounters NexaClip via shared clip link, social post, or search result	Curious, zero commitment	Open Graph preview for shared links shows real generated content to create desire
	Clicks through to nexaclip.com	Slight skepticism	Fast initial page load critical — target < 2s LCP

Homepage (W01)	User Steps	Emotional State	Web UX Consideration
	Lands on homepage. Sees hero: dark navy full-width, large headline, live AI demo widget on right	First impression formed in 3 seconds	Hero must load above the fold at 1280px+ — no scroll required to see the CTA
	Watches demo widget generate a gaming image in real time	Engaged — the demo is real product output	Demo widget auto-runs on load. No interaction required from user
	Scrolls down — reads feature sections	Building understanding	Scroll-triggered animations reveal feature cards
	Reads social proof bar (game logos)	Trust signals land	Social proof bar uses logos not text — scans in <1 second
	Sees pricing teaser at bottom	Price anchoring	Pricing teaser says \$0 to start — removes cost anxiety
	Clicks "Start for free" CTA	Decision to try	CTA button uses --shadow-purple on hover, scale on press

Signup (W05)	User Steps	Emotional State	Web UX Consideration
	Arrives at /signup page	Slight friction (expected)	Signup page has no TopNav — single focus on the form
	Reads form: Email, Username, Password fields	Evaluating effort vs reward	3 fields only. No phone, no DOB, no captcha visible (use invisible reCAPTCHA)
	Optionally uses Google/Apple SSO	Considering the easier path	SSO buttons equally prominent as email form

Signup (W05)	User Steps	Emotional State	Web UX Consideration
	Completes form — taps Create Account	<i>Committed — clicking button</i>	Button loading state: "Creating your account..." + spinner
	Sees email verification note (non-blocking)	<i>Mild anxiety: will this work?</i>	Verification note: "Check your inbox — you can verify later." Never block on email verification
	Redirected into onboarding flow	<i>Relief — moving forward</i>	Immediate redirect — no splash or "you are registered" intermediate screen

J1.2 — Homepage Flow (W01)

W01 Homepage <i>nexaclip.com</i>
ZONE A — TopNav (fixed, 72px): Logo left · Features / Pricing / Blog links centre · "Log in" ghost + "Get started free" primary buttons right.
ZONE B — Hero (100vh min): Dark navy background. Left 7 cols: eyebrow pill, H1 headline (2 lines max), subtitle, 2 CTA buttons, social proof row. Right 5 cols: Live AI demo widget (card with real generation animation).
ZONE C — Social proof bar (80px): Game publisher logos in grayscale. Marquee scroll on loop.
ZONE D — Feature cards (3-up): White cards on gray-50 background. Icon + title + 2-line description + "See how it works →" link each.
ZONE E — Feature deep-dives (alternating): Image Studio (text left, product screenshot right). Creator Coach (screenshot left, text right with "Powered by Claude AI" attribution).
ZONE F — Feed preview (full-width): 6 real content cards from beta. "See what creators are making" heading.
ZONE G — Pricing teaser: 2-column, Free plan details left, pricing card preview right. CTA → /pricing.
ZONE H — CTA band: Navy-900 bg. Centred headline + 2 buttons.
ZONE I — Footer: 4-column. Logo + links + copyright.

J1.3 — Signup Flow (W05) — Detailed Interaction Steps

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	User	Navigates to /signup via homepage CTA or direct URL.	Serve signup page. Check if user already has valid JWT — if yes, redirect to /feed.	<i>Signup page loads. Centred 440px card on gray-50 bg.</i>
2	System	Form renders: Email input, Username input, Password input with strength indicator bar. Google SSO button. Apple SSO button. "Create account" primary button (disabled until form valid).	No backend call yet.	<i>Form default state. Button disabled (40% opacity).</i>
3	User	Types email address in Email field.	No call on every keystroke. Validate format client-side only.	<i>Email field: Default → Filled. No error shown until blur.</i>
4	User	Clicks out of email field (blur).	POST /auth/check-email — debounced 400ms after blur.	<i>If taken: red border + "Email already"</i>

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
			Checks if email is already registered.	<i>registered. Log in →". If available: no change (silent pass).</i>
5	User	Types username. Sees real-time availability check.	GET /auth/check-username?username={val} — debounced 600ms after each keystroke.	<i>Loading: small spinner in field right. Available: green checkmark. Taken: red X + "Username taken". Rules note: "3–20 chars, letters, numbers, underscore."</i>
6	User	Types password. Sees strength bar below field change in real time.	No backend call. Client-side strength calculation only.	<i>Strength bar: red (weak) → amber (fair) → green (strong). Requirement list appears: 8+ chars, 1 number, 1 uppercase. Each checks off as met.</i>
7	User	"Create account" button activates once all fields valid.	No call yet.	<i>Button changes from disabled (40% opacity) to active (full purple, --shadow-purple).</i>
8	User	Clicks "Create account" button.	POST /auth/register. Sends email, username, password. Returns: { accessToken, refreshToken, user }.	<i>Button: "Creating your account..." + spinner replaces text. Button disabled. No layout shift.</i>
9	System	Account created. Non-blocking email verification note: "We sent a verification link to {email}. You can verify later."	Stores tokens in localStorage (nx_access_token, nx_refresh_token). Stores user in NgRx Signal Store. Fires background call to Identity Service to send verification email.	<i>Brief success state on button (green checkmark, 400ms) then immediate navigation.</i>
10	System	Redirects to /onboarding.	Angular Router navigates. No page reload. Previous /signup URL removed from history (replaceUrl: true).	<i>Onboarding page loads. App shell NOT shown — full screen modal-style layout.</i>

J1.4 — Login Flow (W04)

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	User	Arrives at /login. Centred 440px card. "Welcome back" heading. Email + password fields. "Sign in" primary button. "Forgot password?" link. "No account? Sign up →" link.	Serve page. Check JWT — if valid redirect to /feed. If nx_return_to in localStorage, note it for post-login redirect.	<i>Login form default state.</i>
2	User	Enters email and password. Clicks "Sign in".	POST /auth/login. Returns JWT + refresh token or 401 error.	<i>Button: "Signing in..." + spinner.</i>
3	System	Login successful.	Store tokens in localStorage. Set user in NgRx store. Check nx_return_to — navigate to stored URL or /feed.	<i>Fade transition to destination page.</i>

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
4	Error	Wrong credentials: Red banner above form: "Email or password is incorrect." Both fields shake animation (200ms). Password field cleared. Email field retains value.	401 returned. Do not indicate which field is wrong (security best practice).	Error state. User can retry without retyping email.
5	Error	Account locked (5 failed attempts): "Too many attempts. Your account is temporarily locked. Try again in 15 minutes or reset your password."	429 returned with retry-after header. Show countdown timer.	Sign in button disabled with countdown.

J1.5 — Returning User Auto-Login

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	User	Navigates to nexaclip.com. No login action needed.	Angular app initialises. AuthService checks localStorage for nx_access_token. Decodes JWT to check expiry.	App loading (spinner or skeleton for < 500ms).
2	System	JWT is valid and not expired.	User loaded from JWT payload into NgRx store. No network call needed.	Redirect to /feed immediately. No login page shown.
3	System	JWT is expired but refresh token exists.	POST /auth/refresh with refresh token. New access token returned.	Silent background refresh. User never sees login page.
4	System	Both tokens expired or absent.	Clear localStorage tokens. Store current URL as nx_return_to if it was an auth-required page.	Redirect to /login.

Journey 2 — Claude Onboarding Chat

First intelligent interaction · 90 seconds · One-time only

The onboarding chat is the most critical moment in the user lifecycle. It sets the tone for the entire product relationship. It must feel intelligent, fast, and personalised — not like a form with a conversational skin. It runs immediately after registration and never again.

J2.1 — Onboarding Screen Layout

OB Onboarding Chat /onboarding
FULL SCREEN — No app SideNav, no TopBar. Single focus on the conversation.
TOP BAR (48px): NexaClip logo left. Progress indicator centre: 5 unfilled dots, first fills purple as user answers each question. "Skip for now" text link right (small, gray).
LEFT PANEL (40% width): Static panel. Large NexaClip logo/icon. "Your Creator Coach is setting up your account." subtitle. Below: 3 benefit bullets that appear one-by-one as questions progress. Dark navy background. Purple accent.
RIGHT PANEL (60% width): Scrollable chat area. Claude messages (purple-50 bubbles, left-aligned). User answer bubbles (navy-50, right-aligned). Option chips below each Claude question.
BOTTOM (right panel): Input area only visible after Q5 for free-form responses. Not shown during chip-selection phases.
CHIP OPTIONS AREA: Appears below the last Claude message. Full width of right panel. Chips arranged in a responsive grid (2-3 per row). Stagger-animate in from below (80ms delay each).

J2.2 — Detailed Onboarding Flow

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	System	Onboarding page loads. Left panel visible. Right panel shows Claude typing indicator (3 pulsing purple dots in a bubble).	Angular initialises WebSocket connection to Notification Service (/events namespace). POST /coach/onboarding/start — returns opening message.	Left panel visible. Chat area: Claude typing indicator active.
2	Claude	Opening message streams in token by token: "Welcome to NexaClip! I am your Creator Coach. Let me set up your account in about 90 seconds — just answer 5 quick questions." Message finishes. Q1 chip options animate in below.	Server-Sent Events stream tokens via WebSocket coach:token events. Angular component appends each token to the message string.	Message text streams in. After complete, chips animate: stagger 80ms each.
3	System	Q1 chips appear: [Valorant] [Fortnite] [CS2] [Apex Legends] [FIFA] [Minecraft] [Other]. Multi-select enabled. "Continue →" button appears after first selection.	No backend call on chip display.	Chips: default white border. Multi-select possible. Continue button: disabled until ≥1 selected.

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
4	User	Clicks 1–3 game chips. Selected chips turn purple-filled. Clicks "Continue →".	No API call yet — selections held client-side.	<i>Selected chips: purple bg + white text. Continue button activates.</i>
5	System	User answer appears as right-aligned bubble: "Valorant, Fortnite". Progress dot 1 fills purple. Claude typing indicator appears.	POST /coach/onboarding/answer with { question: 1, answer: [...selectedGames] }. System begins building Q2 context.	<i>User bubble slides in from right. Progress dot 1 → filled purple.</i>
6	Claude	Q2 message streams in: "Nice! Who are you making content for?" Q2 chips appear: [Competitive players] [Casual gaming fans] [General gaming audience].	Claude API generates Q2 based on game choices for personalisation. Single-select this round.	<i>Q2 chips animate in. Single-select: clicking one automatically continues (no Continue button needed).</i>
7	User	Clicks one audience option. Auto-submits.	POST /coach/onboarding/answer with { question: 2, answer: selectedAudience }. Progress dot 2 fills.	<i>Selected chip immediately becomes user bubble and submits. No delay — feels responsive.</i>
8	Claude	Q3: "What is your main goal?" Chips: [Grow followers fast] [Build a loyal community] [Attract brand deals].	Auto-submit continues for single-select questions.	<i>Same pattern. Progress dot 3 fills on answer.</i>
9	Claude	Q4: "How often can you realistically post?" Chips: [1–2 times per week] [3–5 times per week] [Daily].	Auto-submit on click.	<i>Progress dot 4 fills.</i>
10	Claude	Q5: "Do you have existing gaming clips or are you starting from scratch?" Options: [I have clips ready] [Starting fresh].	Auto-submit on click. After answer received, POST /coach/onboarding/generate-plan with all 5 answers.	<i>Progress dot 5 fills. All 5 dots now purple.</i>
11	System	Plan generation state: Claude typing indicator returns. Left panel updates: "Building your personalised plan..." with animated pulse.	Claude API generates 7-day plan JSON. ~3–6 seconds. WebSocket streams progress: "Analysing your game niche..." then "Creating your week plan..."	<i>Animated pulse on left panel text. Right panel: extended typing indicator.</i>
12	Claude	Plan reveal: "Your first week is ready! Here is what I have planned for you:" followed by a structured 7-day preview (Mon–Sun cards in a 2-column grid within the chat panel).	Plan JSON stored to users.onboarding_plan in PostgreSQL. Users.onboarding_completed = true.	<i>Plan cards animate in with stagger. Day cards: icon + day + content type + one-line theme.</i>
13	System	"Let's go →" primary button appears below plan. "Explore on my own" ghost link.	No call on button render.	<i>Button pulses subtly to draw attention (CSS keyframe animation, gentle scale 1.0→1.02→1.0).</i>
14	User	Clicks "Let's go →".	Angular Router navigates to /create with query param ?onboarded=true.	<i>Fade transition to Create Hub. Welcome banner at top of Create Hub.</i>

J2.3 — Onboarding Error States

Error	Error	Default	Default
Network lost mid-chat: "You are offline — your progress is saved. Reconnect to continue." All chip answers preserved in sessionStorage.	Claude API timeout (>15s): "This is taking longer than usual. Your answers are saved — tap Retry." Retry resubmits from last answered question.	User clicks "Skip for now": Taken to /feed immediately. Persistent banner at top of every page: "Complete your setup for a personalised experience →"	User refreshes mid-onboarding: If answers stored in sessionStorage, resume from last answered question. If not, restart from Q1 (progress is fast enough this is acceptable).

Journey 3 — Image & Meme Creation

Prompt → AI image → caption → publish · 2–3 minutes

Image creation is the highest-frequency feature and the most important upgrade trigger. The flow must deliver a result that surprises and delights within 8 seconds of clicking Generate. Free users must feel the product working before they hit any paywall.

J3.1 — Image Studio Screen Layout

A03 Image & Meme Studio /create/image
LAYOUT: Two-column persistent split. Left 55%, Right 45%. Both panels stay fixed — no scroll hides either. LEFT PANEL — INPUT: Breadcrumb: Create → Image Studio. Section label: "Describe your image" (label-lg, gray). Prompt Textarea: 5 rows min, 8 rows max, auto-expand. Rotating placeholder every 4 seconds. Character counter bottom-right (0/500). Purple focus ring. Style Picker: "Style" label. 5 horizontal cards: Cinematic / Meme / Pixel Art / Cartoon / Realistic. Icon + label. One always selected. Selected: purple border (2px) + purple-50 fill. Free tier counter: "3 of 5 generations left today · Resets at midnight". Progress dots. Amber if 1–2 left, red if 0. Generate Button: Full-width, 48px, primary purple. "🔮 Generate Image". Disabled when prompt empty. Loading state on click. RIGHT PANEL — RESULT: Empty state (on first load): Dashed border card. Sparkle icon 40px. "Your image will appear here" gray text. Matches the panel dimensions exactly. Loading state: Skeleton card + shimmer animation. "Generating your image..." text below. Result: Generated image fills panel. --radius-xl. AI badge top-left corner. Watermark for free users (semi-transparent logo, bottom-right, 30% opacity). Caption section (loads 0.5s after image): Three option cards (selectable). Editable textarea below selected caption. Character counter. Platform limit badges. Hashtag section: Three bundles (Short/Medium/Long). Active bundle shows removable chips. Action bar (sticky at panel bottom): "Download" ghost button left. "Publish to Feed" primary button right.

J3.2 — Image Generation Flow

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	User	Opens /create/image from Create Hub or direct nav. Left panel loaded with prompt input focused (auto-focus). Right panel shows empty state card.	GET /create/image page. Load user plan data and daily usage count from JWT payload.	Left panel active. Cursor in textarea. Right panel: empty state dashed card.
2	User	Types prompt: "A Valorant Jett agent standing on a Moroccan rooftop at golden hour, cinematic photography". Style "Cinematic" already selected.	No API call during typing.	Character counter increments: "89/500". Generate button activates when text > 0.

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
3	User	Clicks "🔮 Generate Image".	Validate: prompt not empty, daily limit not exceeded (check Redis key rate:{userId}:image_gen:{date}). If valid: POST /content/generate.	Button text → "Generating..." + spinner. Button disabled. Right panel: empty state fades out (200ms), skeleton card fades in.
4	System	Right panel shows animated skeleton with shimmer. "Generating your image... (~6 seconds)" counter below skeleton.	DALL-E 3 API call with style-prefixed prompt. Expected response time: 4–12 seconds.	Skeleton shimmer animation loops. Counter counts up seconds.
5	System	Image arrives. Skeleton fades out (150ms). Generated image fades in (300ms + translateY 8px→0px). AI badge appears top-left. Watermark visible for free users.	Image downloaded from DALL-E URL, watermarked if free tier, uploaded to S3, CDN URL returned. Caption generation begins simultaneously (GPT-4o mini, async ~1s).	Image visible. Button text resets to "🔮 Generate Image". Button re-enables.
6	System	Caption section slides in below image (~1 second after image). "Suggested captions" heading. 3 option cards appear with stagger animation.	GPT-4o mini returns 3 caption variants + 3 hashtag sets. Stored in content record.	Caption cards animate in (stagger 100ms). First card auto-selected (purple border).
7	User	Reviews captions. Clicks caption card 2 to select it.	No API call — client-side selection.	Card 2 gets purple border + purple-50 fill. Card 1 deselects. Selected text copies into editable textarea below.
8	User	Edits caption text in textarea. Selects "Medium" hashtag bundle.	No API call during editing.	Character counter updates. Hashtag chips render below. Platform limit badge: green/amber/red based on platform char limits.
9	User	Clicks "Publish to Feed".	POST /content/{contentId}/publish. Triggers moderation worker (HIGH priority BullMQ queue).	Button: "Publishing..." + spinner. Brief overlay on image: "Reviewing content..." (if moderation >2s).
10	System	Moderation approved (<3s). Success state: brief green flash on Publish button (200ms). Toast notification slides in from top-right: "Your image is live! View on feed →".	Content status → published. Feed Service notified. Analytics event logged.	Toast: "Your image is live!" with teal-700 bg, View link, 4s auto-dismiss.
11	User	Clicks "View on feed →" in toast. OR stays to create another image (prompt field retained).	Angular Router navigates to /feed if user clicks toast link.	If stays: right panel resets to empty state. Prompt field retains last prompt. Generate button re-enabled.

J3.3 — Daily Limit Reached Flow (Free User)

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	User	Clicks "🔮 Generate Image" after using all 5 daily free generations.	POST /content/generate returns 403 with body: { message: "Daily limit reached", limit: 5, upgrade_url: "/upgrade" }.	Generate button shows brief error state.
2	System	Upgrade modal slides up from bottom of screen (not full page redirect). Modal shows: the last generated image (proof of value already received) + "You have created 5 images today — you are on a roll." + Pro benefits list + "\$12/month" pricing + "Unlock unlimited →" CTA + "Maybe later" link.	No backend call for modal display.	Page dims. Modal slides up (350ms spring animation). Backdrop overlay behind modal.
3	User	User clicks "Maybe later". Modal dismisses. Counter now shows "0 generations left · Resets at midnight".	No call.	Modal slides down. Counter badge turns red. Generate button remains disabled with tooltip on hover: "Daily limit reached. Resets at midnight."
4	User	User clicks "Unlock unlimited →". Navigates to /upgrade or Stripe checkout.	POST /billing/create-checkout returns Stripe checkout URL.	Redirect to Stripe checkout in same tab (not new tab — prevents popup blocker).

J3.4 — Image Generation Error States

Error	Error	Error	Loading
DALL-E API timeout (>20s): "Your image is taking longer than usual. We will notify you when it is ready." User can leave — push notification fires when complete.	Content rejected by moderation: Red overlay on generated image. "We could not publish this. [Plain English reason]. Edit your prompt and try again." Prompt pre-filled with previous text.	"Refine" flow: "Not what you wanted? Refine it" link always visible below image. Opens an overlay with the original prompt pre-filled + "What to change" description field. One-tap regeneration with refinement context.	Slow connection: If generation >10s, "Still working on your image..." text replaces the countdown. At 20s: offer to complete in background with notification.

Journey 4 — Gaming Clip Upload & Editing

Upload → Trim → AI Polish → Publish · 5–8 minutes

The clip editor is the most technically complex flow in the product. It involves a large file upload, video playback, timeline interaction, and AI processing pipeline. The UX must manage waiting time with clear progress communication and make the AI polish feel like magic rather than a loading screen.

J4.1 — Three-Step Wizard Structure

Step	Screen	URL	What Happens
Step 1: Upload	Clip Upload (A04a)	/create/clip	User selects video file. Direct S3 upload with XHR progress. Navigates to editor when upload complete.
Step 2: Trim	Clip Editor — Trim (A04b)	/create/clip/:id/edit?step=trim	User defines highlight window on filmstrip timeline. Sets trim handles. Previews selection.
Step 3: Polish & Publish	Clip Editor — Polish (A04b)	/create/clip/:id/edit?step=polish	Whisper transcription. Caption editing. Music selection. Final publish.

J4.2 — Upload Flow (Step 1)

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	User	Clicks "Gaming Clip" from Create Hub. Navigates to /create/clip. Upload zone: large dashed-border area (320px height). "Drop your gaming clip here" with upload icon. "or click to browse" below. File format and size limits shown.	GET /create/clip page. No API call yet.	Upload zone in default state.
2	User	Drags a video file onto the drop zone or clicks "Browse" and selects from file picker.	No call yet. Client-side validation: check file extension (mp4/mov/avi) and file size (500MB free / 5GB Pro).	On valid drag-over: dashed border turns solid purple, bg turns purple-50. On drop: validation runs.
3	System	File validation passes. Upload zone transforms: shows filename, file size, progress bar (12px height), "Uploading... 0%" + estimated time, "Cancel" link.	POST /content/upload-url — returns pre-signed S3 URL + contentId. Then XHR PUT to S3 URL directly from browser.	Progress bar fills in real time based on XHR progress events.
4	System	Upload completes (progress bar reaches 100%). Brief "Upload complete ✓" state	S3 Lambda notifies Content Service: upload complete. Content record status → "uploaded". Frame thumbnail extraction job begins.	Progress bar fills green. Checkmark. Then fade transition to /create/clip/:id/edit?step=trim.

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
		(green check, 600ms). Auto-navigates to trim editor.		
5	Error	File too large (free tier): Inline message below upload zone: "This file is 1.8GB — free accounts support up to 500MB. Trim the recording first, or upgrade to Pro for up to 5GB." Upgrade link inline.	Client-side check — no upload attempted.	Upload zone shows error state (red dashed border). Message below.
6	Error	Upload interrupted (network): "Upload interrupted. Tap Retry to resume where you left off." Retry uses S3 multipart upload resume from last committed part.	Multipart upload metadata stored server-side. Resume continues from last successful part.	Progress bar freezes. Error banner above progress bar. Retry button appears.

J4.3 — Trim Editor Flow (Step 2)

A04b Clip Trim Editor `/create/clip/:id/edit?step=trim`

STEP INDICATOR: Horizontal 3-step bar at page top. Step 1 (Upload): completed checkmark. Step 2 (Trim): active/purple. Step 3 (Polish): upcoming/gray.

VIDEO PLAYER: 16:9 ratio. Fills content area max-width minus padding. Standard controls: play/pause/mute/volume/fullscreen. Plays only within trim selection during trim mode.

TIMELINE STRIP (below video, full content width, 72px height):

Frame thumbnails: horizontal filmstrip of extracted frames (progressive load from left).

Left trim handle: orange pill (16px wide, full height). Drag to set clip start.

Right trim handle: same. Drag to set clip end.

Purple overlay: between handles at 30% opacity shows selected region.

Playhead: thin purple vertical line + 12px circle top. Draggable.

Time markers: every 10 seconds below strip.

TRIM CONTROLS ROW (below timeline):

Left: "Start time" editable input (e.g. 0:14). "End time" editable input (e.g. 0:37).

Centre: "Duration: 0:23" badge. Updates in real time.

Right: "Preview selection" button. "Apply Trim & Continue →" primary button.

DURATION LIMIT: Free: "0:23 / 1:30 max" shown in badge. Pro: "0:23 / 3:00 max".

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
7	System	Trim editor loads. Video player shows first frame. Timeline strip begins rendering frame thumbnails from left (progressive). Frame thumbnails appear as they load.	Frame thumbnails extracted every 2 seconds by FFmpeg worker. CDN URLs returned progressively.	Timeline: skeleton thumbnails → real frames appearing left to right.
8	User	Drags left trim handle to set start point. Video jumps to that timestamp. Duration badge updates.	No API call. Client-side only.	Handle position updates. Selection overlay redraws. Duration badge: real-time update.

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
9	User	Drags right trim handle to set end point. Duration badge shows final selection.	No call.	Selection region between handles highlighted. Duration badge: "0:28".
10	User	Clicks "Preview selection". Video plays from trim start to trim end only. Looping.	No call. HTML video element currentTime set to trim start. playbackRate 1x.	Video plays within selection bounds. "Stop preview" button replaces play button.
11	User	Satisfied with selection. Clicks "Apply Trim & Continue →".	PATCH /content/:id/trim — { trimStart: 14.2, trimEnd: 42.6 }. Content record updated. Dispatch transcription job to BullMQ.	Button: "Saving..." + spinner (500ms). Then fade transition to polish step.

J4.4 — AI Polish & Publish Flow (Step 3)

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
12	System	Polish step loads. Left panel: "Polishing your clip" heading. 3-step progress list with animated icons. Right panel: empty preview placeholder.	BullMQ transcription worker picks up job. Whisper API call begins.	Step 1 (Transcribing): spinner icon + "Finding the words in your clip..."
13	System	Step 1 completes: checkmark replaces spinner on step 1. Step 2 activates.	Whisper returns timestamped transcript. VTT caption file generated. Stored to S3.	Step 1: green checkmark. Step 2: spinner + "Adding captions to your clip..."
14	System	Step 2 completes. Step 3 activates.	Caption file stored. Peak audio timing analysed. Music library matched to content genre.	Step 2: green checkmark. Step 3: spinner + "Matching effects..."
15	System	All 3 steps complete. "Polish complete!" heading replaces progress list. Results slide up into left panel.	Content status → "ready_to_publish". WebSocket emits content:status update.	Results section: captions list, effects toggle, music carousel.
16	User	Reviews captions list (transcript lines with timestamps). Taps a caption line to edit it inline.	No API call during edit.	Clicked line: text becomes editable input. Blue border. Save on blur or Enter.
17	User	Browses music carousel: 10 tracks shown with genre tags. Clicks play icon on a track.	No API call. HTML Audio element plays 15-second preview from S3.	Track preview plays. Play → pause button. Track row highlighted.
18	User	Clicks "Use This Track". Music selection confirmed.	PATCH /content/:id/music — { musicTrackId }.	Music badge appears on right panel preview.
19	User	Clicks "Preview" on right panel. Full video preview with captions overlaid.	No API call. Constructed client-side from clip CDN URL + VTT file.	Preview plays. Captions render using HTML5 <track> element.
20	User	Enters clip title (required). Optional description.	No API call on typing.	Character counters update. Title: 80 char max. Description: 300 char max.
21	User	Clicks "Publish to Feed".	POST /content/:id/publish. Moderation check queued.	Button: "Publishing..." + spinner.

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
22	System	Moderation approved. "Your clip is live! 🎉" full-width success banner. Confetti animation from clip thumbnail.	Content published. MediaConvert final transcode job queued for HLS streaming.	Confetti burst (CSS, 1.2s). Success banner. Links: "View on Feed", "Create Another".

Journey 5 — Social Feed & Engagement

Daily content consumption and community interaction

J5.1 — Home Feed Layout & Initial Load

A01 Home Feed /feed
APP SHELL: SideNav (240px left, collapsible). TopBar (64px top, extends right of nav). Content area fills remaining viewport.
FEED HEADER (sticky below TopBar): "Home" H2 left. "Following / Discover" toggle tabs right. "🔥 Create" primary button far right.
TWO-COLUMN LAYOUT: Left 8 cols (feed). Right 4 cols (sidebar widgets).
LEFT — CONTENT FEED: Masonry-style 2-column card grid. Each ContentCard: 16:9 thumbnail, user info row (avatar + username + game badge + time), caption (2 lines), engagement row (heart + count, comment + count, share, bookmark).
RIGHT — SIDEBAR WIDGETS: Widget 1: "Trending Today" — today's trend brief from Claude. Game icon + 1-sentence insight + "Create from this →" button. Widget 2: "Your Stats (7 days)" — 3 stat pills: Views, Likes, Shares. "Full report →" for Pro / "Unlock insights →" for free. Widget 3: "Suggested Creators" — 4 rows: avatar + name + game badge + "Follow" button. Widget 4 (free users only): Upgrade nudge card — "Creator Coach is waiting for you. Upgrade to Pro →".

J5.2 — Feed Load & Scroll Flow

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	User	Navigates to /feed or clicks Home in SideNav.	GET /feed?limit=20. Check follows count. <5 follows: discovery feed. 5+ follows: personal feed.	Feed skeleton: 4 ContentCard skeletons (2×2) visible. Sidebar widgets load in parallel.
2	System	First 20 posts load. Skeleton cards replaced by real content cards with stagger animation (50ms delay per card).	Posts hydrated: content data + user info + engagement counts + is_liked_by_me check.	Cards snap into view sequentially. Feed feels alive.
3	User	Scrolls down the feed. Reads content, watches clips.	When user reaches post 15 of 20: prefetch next 20 posts in background (GET /feed?cursor={id}).	No loading indicator during prefetch. Seamless infinite scroll.
4	System	Next 20 posts silently appended below existing cards. User never sees a load break.	Response appended to existing posts array in NgRx store.	Feed continues smoothly. No visual interruption.
5	User	Clicks on a video clip thumbnail.	Navigate to /feed/post/:id. Load full post data.	Slide transition (post detail slides in from right, feed remains behind). Video auto-plays muted.

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
6	User	Scrolls back to top and pulls down (overscroll on desktop: scrolls to negative position).	GET /feed?limit=20 (fresh fetch).	<i>Pull-to-refresh indicator appears at top. New posts prepended with "3 new posts" badge.</i>

J5.3 — Engagement Interactions

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
7	User	Clicks heart icon on a post.	POST /content/:id/like — toggles like. Optimistic update: do not wait for response.	<i>Heart: fills immediately (purple) + count increments by 1. Bounce animation (scale 1→1.3→1.0). If already liked: unfills, count decrements.</i>
8	System	Server confirms like toggle.	Like recorded in content_likes table. content_engagement_counts updated. Redis counter incremented.	<i>No visible change (optimistic was correct). On server error: heart reverts and toast: "Could not save your like. Please try again."</i>
9	User	Clicks comment icon. Comment panel slides out from right side of the post (within the post detail view). Existing comments listed.	GET /content/:id/comments?limit=20.	<i>Comment panel: comments load with skeleton → content. Input field at bottom with Send button.</i>
10	User	Types comment in input field. Clicks "Send".	POST /content/:id/comment — { body: text }.	<i>Comment appears immediately at top of comments list (optimistic). Send button: spinner during call.</i>
11	User	Clicks "Follow" button on a post's creator avatar.	POST /users/:id/follow.	<i>"Follow" → "Following ✓" immediately (optimistic). Button: white border → teal fill. Reverting shows "Follow" again.</i>
12	User	Clicks share icon. Native browser share sheet opens (navigator.share()). If not supported: clipboard copy fallback.	Generate shareable link with UTM params: nexaclip.com/p/{contentId}?ref={userId}.	<i>On share: "Link copied!" toast (2s). On native share: OS native sheet opens.</i>

J5.4 — Empty Feed & Discovery

Empty	Empty	Loading	Error
-------	-------	---------	-------

New user (0 follows): Show discovery feed automatically. "Suggested Creators" card appears at position 4 in feed. Team-curated content fills feed.	User follows creators with no posts: "Nothing new from your follows. Here is what is trending →" with CTA to discover tab.	Initial feed load: 4 skeleton ContentCards (2-column, matching card dimensions exactly). Sidebar: 3 widget skeletons.	Feed load fails: "Could not load your feed. Check your connection and try again." Retry button. Last cached feed shown below (if available).
---	--	--	--

Journey 6 — User Profile Management

Viewing, editing, and managing creator profiles

J6.1 — Own Profile View (A09)

A09 Own Profile /profile
COVER IMAGE: Full-width 1200×280px. Default: gradient using game niche colors. Upload button appears on hover (edit mode).
PROFILE HEADER: Avatar (88px circle) overlaps cover bottom-left (offset -24px). Display name (H2). Username (@handle, gray). Plan badge (FREE/PRO/STUDIO pill). Edit Profile button top-right.
STATS ROW: 3 stat pills: Posts (count) · Followers (count, tappable → follower list) · Following (count, tappable → following list). Engagement rate (Pro only, else locked icon).
GAME NICHE CHIPS: Horizontal chip row showing selected games. Teal outline chips.
BIO: 1–3 lines. "Add a bio →" link if empty.
CONTENT GRID: 3-column grid of ContentCard thumbnails. Sticky tab bar: Posts Liked. "Create your first post →" empty state if no content.
COMPLETION BANNER (if onboarding incomplete): Amber-50 bg. "Complete your setup for personalised recommendations" + "Finish setup →" link.

J6.2 — Edit Profile Flow (A10)

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	User	Clicks "Edit Profile" button on own profile. Navigates to /profile/edit.	GET /users/me — pre-fills form with current data.	Edit profile form loads. Centred card layout (640px max). Fields pre-filled.
2	User	Clicks on avatar. Upload zone overlay appears on the avatar circle.	No call yet.	Avatar: dashed border + "Change photo" overlay text on hover.
3	User	Selects image file. Crop modal opens (square crop, 1:1 ratio).	No API call during crop.	Crop modal: image preview with movable/scalable crop frame.
4	User	Confirms crop. Avatar preview updates immediately.	Upload cropped image to S3 via pre-signed URL.	Avatar preview shows new image with upload progress ring around it.
5	User	Updates Display Name and Bio fields.	No API call on typing.	Character counters: Display Name 50 char, Bio 200 char.
6	User	Updates Game Niche: tag multi-select input. Types "val" — autocomplete shows "Valorant". Clicks to add.	No API call — client-side filtering of game list.	Tags appear as pills inside the input. Each has an × to remove. Max 5 tags.
7	User	Clicks "Save Changes".	PATCH /users/me — { displayName, bio, gameNiche, avatarUrl }. Returns updated user.	Button: "Saving..." + spinner. On success: navigate back to /profile.

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
8	System	Profile page shows updated data. Toast: "Profile updated successfully."	NgRx auth store updated with new user data.	<i>Profile page with new avatar + bio visible.</i>

J6.3 — Viewing Another User Profile (A08)

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	User	Clicks on another user's avatar or username anywhere in the app.	Angular Router navigates to /users/:id.	<i>Profile page loads. Structure same as own profile but: Follow button replaces Edit button. No edit overlays. Liked tab hidden.</i>
2	User	Clicks "Follow" button.	POST /users/:id/follow.	<i>Button: "Follow" → "Following ✓" (teal fill). Follower count on their profile +1.</i>
3	User	Clicks a post thumbnail in the content grid.	Angular Router navigates to /feed/post/:id.	<i>Post detail slides in from right.</i>

Journey 7 — Analytics Dashboard & Weekly Report

Performance insight for free and Pro users

J7.1 — Analytics Dashboard Layout (A05)

A05 Analytics Dashboard /analytics
LAYOUT: Full content area. Max-width 1200px. No right sidebar column (analytics needs full width).
PAGE HEADER: "Analytics" H1 left. Date range dropdown right: "Last 7 days ▾". For Pro users on Mondays: amber banner "Your weekly report is ready → View report".
STAT CARDS ROW (4 cards): Total Views · Total Likes · Total Shares · Engagement Rate. Each: icon (24px), large number (display-md), label, trend delta (↑8% teal or ↓3% red). Engagement Rate card: locked for free users (frosted blur + lock icon + "Pro feature" label).
CHARTS SECTION (2 charts): Left (8 cols): "Views over time" line chart. Right (4 cols): "Content breakdown" donut. Free: line chart shows 3 days only. Rest blurred with "See 30-day history with Pro →" overlay.
BEST POSTING TIME (full width, Pro only): Heat map grid 7 days × 24 hours. Purple intensity scale. Free: fully blurred with "Unlock with Pro" centred overlay.
TOP POSTS TABLE: Thumbnail · Title · Type · Views · Likes · Engagement Rate · Published. Sortable headers. Hover row: "View analytics" + "Share" inline actions.
WEEKLY REPORT CARD (Pro only): Full-width purple-50 card. Claude-written narrative. 4 sections. Each action item has "→ Create from this" link.

J7.2 — Analytics Flow — Free User

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	User	Navigates to /analytics.	GET /analytics/metrics?range=7d. Plan check from JWT.	<i>Skeleton: 4 stat card skeletons, 2 chart skeletons.</i>
2	System	Metrics load. Free user sees: 4 stat cards (Views, Likes, Shares, Engagement Rate). Engagement Rate has frosted blur overlay + lock icon.	Metrics returned. Engagement Rate value intentionally included in response — just blurred client-side.	<i>Cards animate in (stagger 80ms). Engagement Rate: blur filter applied.</i>
3	User	Clicks on the locked Engagement Rate card.	No API call. Upgrade sheet triggered client-side.	<i>Upgrade sheet slides up from bottom. Shows: engagement rate value blurred behind sheet, Pro benefit context.</i>
4	User	Scrolls to Weekly Report card. Sees 2 lines of real report text then blur overlay. "Your Week in Review" heading visible.	GET /analytics/report/latest returns null for free user. Show placeholder report with actual structure.	<i>Report structure visible. Content blurred after line 2. "Unlock with Pro to read your full report" CTA centred.</i>
5	User	Clicks "Unlock with Pro →". Upgrade sheet opens.	Navigates to upgrade flow (see Journey 10).	<i>Upgrade sheet. Feature context: "Weekly Claude AI report with your full performance analysis."</i>

J7.3 — Analytics Flow — Pro User

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
6	User (Pro)	Navigates to /analytics. All 4 stat cards show full data. Line chart shows 30-day history. Heat map visible and interactive.	GET /analytics/metrics?range=30d — extended range for Pro.	Full analytics visible. No blur overlays. All data accessible.
7	System	Monday morning: amber banner at top: "Your weekly report is ready → View report". Notification badge on Analytics SideNav item.	GET /analytics/report/latest returns this week's Claude-written report.	Banner visible. Badge on SideNav.
8	User (Pro)	Clicks "View report". Full report card expands in-page.	Report already loaded — no additional call.	Report section scrolls into view. Card expands smoothly.
9	User (Pro)	Reads action item: "Post a short Valorant clip on Wednesday at 7pm." Clicks "→ Create from this".	Angular Router navigates to /create with query param: ?context=Wednesday+clip+Valorant.	Create Hub loads. Context banner at top: "From your weekly plan: Post a Valorant clip Wednesday 7pm."
10	User (Pro)	Hovers over heat map cell (e.g. Wednesday 7pm).	No API call.	Tooltip appears: "Wednesday 7pm — Your highest engagement window (avg 3.2× above your baseline)."

J7.4 — Analytics Error & Edge States

Empty	Empty	Loading	Error
New user with no posts: illustration + "Publish your first post to start seeing analytics" + "Create now →" button.	Pro user, no report yet (first week): "Your first weekly report will be ready next Monday. Keep creating content this week."	Metrics loading: 4 stat card skeletons + 2 chart area skeletons exactly matching real content dimensions.	Analytics API error: "Could not load your analytics. We are working on it." Retry button. Last cached data shown if available with "Showing cached data from [date]" notice.

Journey 8 — Creator Coach (Claude Chat)

AI-powered coaching for growth strategy

The Creator Coach is the flagship Pro feature. Its UX must balance the intelligence of Claude with the warmth of a genuine advisor. The streaming response feels like a colleague thinking and responding in real time — not a chatbot.

J8.1 — Coach Layout (A06)

A06 Creator Coach /coach
LAYOUT: Full content area. Two-column. Left 280px (conversation history). Right: main chat area. FREE USER GATE (shown before chat): Centred single-column overlay within the page. Claude icon 80px. "Meet your Creator Coach" H2. 3 benefit bullets. "3 free messages remaining" info badge (amber). "Start a conversation" primary CTA. "Upgrade to Pro for unlimited" text link. LEFT PANEL — History: "New conversation" button top. List of previous conversations: date + first message excerpt. Active highlighted purple-50. Scrollable. RIGHT PANEL — Chat area: HEADER (sticky): "Creator Coach" title. "Powered by Claude AI (Anthropic)" attribution badge (small, gray). Free users: "2 messages remaining this month" amber counter top-right. CONTEXT PILLS (first message only): Small chips showing known context: "Game: Valorant" · "Goal: Grow followers" · "Last week: 1.2K views". Tappable to update. MESSAGES: Scrollable area. Full height minus header and input. INPUT AREA (sticky bottom): Multi-line textarea (1 row expanding to 4). Placeholder: "Ask your Coach anything...". Send button (primary icon). Keyboard shortcut: Ctrl+Enter. SUGGESTED PROMPTS (empty state only): 3 chip suggestions: "What should I post this week?" / "Why are my views dropping?" / "Analyse my best-performing content"

J8.2 — Coach Chat Flow (Pro User)

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	User	Navigates to /coach. Left panel shows conversation history (or empty state for new users). Right panel shows chat area with context pills.	GET /coach/conversation — loads conversation history (messages JSONB array). GET /analytics/summary?days=14 — loads metrics context. GET /trends/brief?niche={userGameNiche} — loads today's trends.	Page loads. Context pills show personalised tags. Chat history loads.
2	User	Clicks "New conversation" or types directly in input. Types: "My views dropped 40% this week. What happened?"	No API call on typing.	Input field grows to 2 rows as text is entered.
3	User	Presses Ctrl+Enter or clicks Send.	POST /coach/message — { message: text, conversationId }. Angular subscribes to WebSocket room for this user.	User message bubble appears immediately (right-aligned, navy-50 bg). Input clears. Send button disabled.

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
4	System	Claude typing indicator appears in chat (left-aligned, purple-50 bubble, 3 animated dots).	BullMQ coach.worker picks up job. Builds context: system prompt + last 30 messages + user metrics + trend brief. Claude API streaming call begins.	<i>Typing indicator: 3 dots pulse sequentially.</i>
5	Claude	Text begins streaming into the Claude bubble. Each token appears as it arrives: "Looking at your data from the past 14 days..."	WebSocket emits coach:token events. Angular appends each token to the message string in real time.	<i>Text streams character by character. Cursor blink at end of text.</i>
6	Claude	Full response delivered. Example: "Your views dropped because your last 5 posts were over 45 seconds long. Your audience strongly prefers clips under 30 seconds — your best post this month was 22 seconds. This week, aim for 20–28 second clips. Your Wednesday 7pm slot is still your strongest — use it."	WebSocket emits coach:complete with full message. AI job record updated with token cost.	<i>Cursor disappears. Response complete. Send button re-enables.</i>
7	User	Replies: "Can you write me some clip ideas for this week?"	Same pattern. Previous messages included in context (last 30 messages).	<i>Conversation continues. User message bubble. Claude streaming response.</i>
8	System	Conversation auto-saved. Left panel history updates with this conversation's first message as the label.	Conversation messages JSONB updated in PostgreSQL after each exchange.	<i>Left panel: new conversation item at top with first message preview.</i>

J8.3 — Free User Coach Experience

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
9	User (Free)	Navigates to /coach. Sees gate screen: "Meet your Creator Coach". "3 free messages remaining" badge.	Check coach_message_count for this month from Redis.	<i>Gate screen with 3 available messages clearly communicated.</i>
10	User (Free)	Clicks "Start a conversation". Gate screen fades out. Chat interface loads.	No API call. Just UI state change.	<i>Chat interface with amber counter badge: "3 messages remaining this month".</i>
11	User (Free)	Sends 1st message. Gets Claude response. Counter → "2 messages remaining".	Usage tracked in Redis: rate:{userId}:coach_msgs:{YYYY-MM} increments.	<i>Counter badge updates after each exchange.</i>
12	User (Free)	After 3rd message is sent and response received. Input field locked. Upgrade prompt appears:	4th message attempt returns 403.	<i>Input disabled. Prompt overlay: "You have used your 3 free messages this month. Upgrade to Pro for unlimited coaching." Two CTAs: "Upgrade to Pro" +</i>

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
				"Come back next month".

J8.4 — Coach Error States

Error	Error	Loading	Empty
Claude API timeout (>15s): "Your Coach is taking a moment to think. This sometimes happens with complex questions." Still streaming if possible; else: "Try asking a shorter question or come back in a moment."	Network lost during streaming: Partial response shown with "(Connection lost — response may be incomplete)". Retry button to re-ask the same question.	Initial conversation load: Skeleton for last 5 message pairs. Maintains scroll position.	First ever conversation: Welcome message from Claude already shown: "Hi! I'm your NexaClip Creator Coach. I know your game niche and your recent performance. What would you like to improve?" + 3 suggested question chips.

Journey 9 — Free to Pro Upgrade

Upgrade moments, Stripe checkout, and plan activation

The upgrade journey is triggered from multiple places in the product — always in context of the feature a user was trying to use. The upgrade sheet approach means users never leave the page they were on.

J9.1 — Upgrade Trigger Points

Trigger Location	Trigger Event	Upgrade Sheet Context
Image Studio (/create/image)	Daily limit (5 images) reached on Generate click	Shows last generated image + "Unlimited generations with Pro"
Analytics (/analytics)	Clicks locked Engagement Rate card	Shows engagement rate preview blurred + "Unlock full analytics"
Analytics (/analytics)	Clicks blurred Weekly Report section	Shows report preview + "Get your Claude AI weekly coaching report"
Creator Coach (/coach)	4th message in a month (free limit)	Shows conversation context + "Unlimited coaching conversations with Pro"
Content Library (/my-content)	Uploads clip over 500MB (free limit)	Shows file size indicator + "Upload up to 5GB with Pro"
Any page	Clicks "Upgrade" in SideNav bottom banner	General upgrade sheet with full feature comparison
/upgrade page	Direct navigation or deep link	Full pricing page (same as W03 but in app context)

J9.2 — Upgrade Sheet Flow (In-Context)

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	System	Upgrade sheet slides up from bottom of screen. Page dims (backdrop overlay rgba(0,0,0,0.5)). Sheet height: 70vh. Never full screen.	Render upgrade sheet with feature-specific context from the trigger point.	Sheet slides up with spring animation (350ms). Page scroll locked.
2	System	Sheet content structure: 1. Hero section: feature they were using + what they gain. 2. Feature benefit list: 8 Pro features with purple checkmarks. 3. Pricing: "\$12/month" large text. Annual toggle: "Save 17% — Pay annually". 4. "Start Pro now →" primary full-width CTA. 5. "Maybe later" text link below button.	No API call to render sheet.	Sheet renders fully with all content visible (no additional scrolling needed within sheet).
3	User	Clicks the annual/monthly toggle.	No API call. Price display updates: \$12/mo → \$10/mo equivalent.	Price text updates smoothly with number transition animation.

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
4	User	Clicks "Start Pro now →".	POST /billing/create-checkout — { planId: "pro_monthly" or "pro_annual" }. Returns Stripe checkout URL.	Button: "Opening checkout..." + spinner. Then redirect to Stripe.
5	System	Stripe checkout opens in SAME TAB (no new window — prevents popup blockers). Stripe-hosted page. Email pre-filled from user account.	Stripe Checkout Session created server-side.	Full page navigation to Stripe checkout. Standard browser back button returns to NexaClip.
6	User	Completes payment on Stripe. Redirected back to NexaClip /upgrade/success?session_id=xxx.	Stripe webhook fires: subscription.created. Identity Service: update user.plan = "pro". New JWT issued. Push notification sent: "You are now Pro!".	Upgrade success page loads.
7	System	Success page: confetti animation. "Welcome to Pro! 🎉" H1. List of newly unlocked features animating in (stagger). "Go back to what you were doing →" primary CTA (returns to the page where upgrade was triggered).	Angular fetches updated user via GET /auth/me. Updates NgRx store with new plan.	Confetti 2 seconds. Features unlock reveal animation. CTA returns user to exact location.
8	User	Clicks "Go back to what you were doing →". Returns to /create/image (or wherever upgrade was triggered).	nx_return_to stored in sessionStorage when upgrade sheet opened. Used to return.	Returned to exact page. All locked features now unlocked. No more counter badges. No blur overlays.

J9.3 — Plan Downgrade / Cancellation Flow

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
9	User	Goes to /settings. Clicks "Billing" sub-section. Sees: current plan, next billing date, "Cancel subscription" link.	GET /billing/status — returns subscription details from Stripe.	Billing settings page.
10	User	Clicks "Cancel subscription". Confirmation dialog: "Are you sure? You will keep Pro access until [date]. Your content, analytics, and conversations are not deleted."	No API call yet.	Confirmation modal. Two buttons: "Keep my Pro plan" (primary) + "Yes, cancel" (destructive red text button).
11	User	Confirms cancellation.	POST /billing/cancel — calls Stripe subscription cancel at period end.	Success toast: "Subscription cancelled. You have Pro access until March 31." Billing page updates to show "Cancels on March 31."
12	System	On expiry date: user plan changes to free. Next login re-issues JWT with plan: "free". Pro features locked.	Stripe webhook: subscription.deleted. Update user.plan → "free". New JWT on next login.	Upgrade prompts reappear. Features lock. No abrupt notification — gentle re-introduction of free limits.

Journey 10 — Content Library & Settings

Managing published content and account settings

J10.1 — Content Library (A11)

A11 Content Library /my-content
LAYOUT: Full content area. Table-style list view.
FILTER BAR: "All" (default) / "Clips" / "Images" / "Drafts" / "Rejected" tab filters. Sort dropdown: "Newest first" / "Most viewed" / "Engagement". "Bulk select" checkbox in header row.
TABLE HEADER: Checkbox · Thumbnail · Title · Type badge · Status badge · Views · Likes · Created date · Actions (...).
TABLE ROWS: Each row: 60px height. Thumbnail 80×45px. Title truncated to 1 line. Type: "Clip" or "Image" badge. Status: green "Published" / amber "Draft" / gray "Processing" / red "Rejected". View count. Like count. Formatted date.
ROW HOVER: "View analytics" link appears. "Share" link appears. "..." menu shows: Edit, Delete, Republish (if rejected).
BULK SELECT MODE: Checkbox column activates. Bulk action bar appears at bottom: "Delete selected" (red) / "Export links".
EMPTY STATE: Illustration. "Nothing here yet. Create your first post." Create CTA.

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	User	Navigates to /my-content. Table loads with last 30 items.	GET /content?userId=me&limit=30&sort=created_desc.	Table skeleton (8 rows × 5 cols) → real content with stagger animation.
2	User	Clicks "Rejected" filter tab.	GET /content?userId=me&status=rejected.	Table reloads with only rejected content. If empty: "No rejected posts — your content looks good!" with checkmark illustration.
3	User	Clicks ... on a rejected post. Dropdown: Edit / Delete / Republish.	No call on menu open.	Dropdown menu appears (shadow-lg card).
4	User	Clicks "Republish". Navigates back to the content editor with rejection reason shown as a banner.	GET /content/:id — pre-fills editor with existing content data.	Image Studio or Clip Editor opens. Amber banner: "Previously rejected: [reason]. Make edits and republish."
5	User	Clicks delete (... → Delete). Confirmation dialog.	No call yet.	Modal: "Delete this post? This cannot be undone." Confirm / Cancel.
6	User	Confirms delete.	DELETE /content/:id — soft delete (status → deleted, deleted_at timestamp).	Row disappears from table with

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
				<i>fade animation.</i> <i>Toast: "Post deleted."</i>

J10.2 — Settings (A13)

Settings Section	Flow Description
Account Settings	/settings/account: Change display name, email, and password. Email change requires verification of new address. Password change requires current password. All saved with PATCH /users/me.
Notification Settings	/settings/notifications: Toggle switches for each notification type: New follower, Post liked, Comment received, Weekly report ready, Trend brief. Preferences saved to user_preferences table.
Billing	/settings/billing: Shows current plan, next billing date, invoice history (last 6 invoices as links). Cancel subscription link. Upgrade CTA if on free plan.
Privacy	/settings/privacy: Toggle: Public profile / Private profile. Public feed visibility. Data download request button (GDPR). Cookie preferences.
Danger Zone	/settings/account#danger: "Delete my account" section with red border. Requires typing username to confirm. Shows consequences (all content deleted within 30 days). Irreversible. POST /users/me/delete.

11. Cross-Journey System Flows

These flows run automatically in the background across all journeys. They are not triggered by explicit user navigation but by system events.

11.1 — Content Moderation Flow (All Content Types)

Every piece of content goes through this flow automatically when the user taps Publish, regardless of content type (image, clip, meme).

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	User	Taps "Publish to Feed" on any content creation screen.	POST /content/:id/publish queues moderation job (HIGH priority BullMQ).	<i>Publish button: "Publishing..." + spinner.</i>
2	System	Moderation check running (<3 seconds). If <2 seconds: only the button spinner is visible — no separate UI state.	Claude API call with MODERATION_SYSTEM_PROMPT. Returns { decision, reason, confidence }.	<i>If <2s: only button spinner. If 2–3s: "Reviewing content..." overlay appears on the content preview.</i>
3	System	Decision: APPROVED. Success flow activates.	content.status → published. Feed notified. Analytics event logged: content.published.	<i>Button success state (green flash 200ms). Toast: "Your [image/clip] is live!" with "View on feed →" link.</i>
4	System	Decision: FLAG_FOR_REVIEW. Published anyway. Human review queue populated.	content.status → pending_review. Human reviewer notified.	<i>Same as APPROVED from user perspective. Small "Under review" dot on post visible only to the post owner.</i>
5	System	Decision: REJECTED. Rejection screen overlays the content.	content.status → rejected. Reason stored.	<i>Current creation screen: orange shield icon + "We could not publish this." heading. Plain-English reason. "Edit and try again" primary button. "Remove this post" text link.</i>
6	User	Clicks "Edit and try again". Returns to creation screen with reason banner.	Content status remains "rejected" until next publish attempt.	<i>Creation screen with amber banner: "Rejected: [reason]. Edit to address this."</i>
7	User	Makes edit. Re-taps Publish.	New moderation check. Fresh Claude API call. Previous rejection context not included (evaluated fresh).	<i>Same flow from Step 1.</i>

11.2 — Toast Notification System

Trigger	Type	Message Copy	Duration
Content published	Success (teal)	Your image is live! View on feed →	4 seconds
Content rejected	Error (red)	We could not publish this. See why →	6 seconds (longer for important info)
Like action confirmed	Info (subtle gray)	(No toast — like is silent, optimistic UI only)	—
Follow confirmed	Success (teal)	(No toast — button state change is sufficient)	—
Profile saved	Success (teal)	Profile updated.	3 seconds
Upload complete	Success (teal)	Upload complete — opening editor.	2 seconds (then auto-navigate)
Upload error	Error (red)	Upload failed. Check your connection and retry.	6 seconds + Retry button
Plan upgraded	Success (teal)	Welcome to Pro! All features unlocked.	6 seconds
Coach message limit	Warning (amber)	You have used your 3 free messages this month. Upgrade for unlimited.	6 seconds + Upgrade link
Network offline	Warning (amber)	You are offline — some features may be unavailable.	Persistent until reconnection
Network restored	Success (teal)	Back online.	2 seconds

11.3 — Push Notification → App Navigation

Notification Type	Deep Link Target	User Experience on Tap
Weekly report ready (Pro, Monday 8am)	nexaclip://analytics/report	Opens /analytics with report section scrolled into view and highlighted.
New follower	nexaclip://users/:followerId	Opens the follower's profile page.
Comment on your post	nexaclip://feed/post/:postId#comments	Opens post detail with comments panel expanded.
Content published (after background processing)	nexaclip://my-content	Opens content library. New post highlighted with pulsing ring for 3 seconds.
Trend brief available (daily 8am)	nexaclip://feed	Opens feed. Trending Today widget highlighted with purple border for 5 seconds.
Coach message response (if app was closed)	nexaclip://coach/:conversationId	Opens Coach with the specific conversation scrolled to the new response.

11.4 — Session Expiry & Token Refresh

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	System	User is using the app. Access token expires (1-hour TTL).	Angular HTTP interceptor receives 401 response on any API call.	<i>User sees nothing unusual — intercept is transparent.</i>
2	System	Interceptor attempts silent refresh.	POST /auth/refresh with refresh token from localStorage.	<i>The failed request is queued. Refresh proceeds.</i>
3	System	Refresh succeeds. Original request is retried automatically.	New access token stored. Original request retried with new token.	<i>User never sees a login prompt. Feature works normally.</i>
4	System	Refresh fails (refresh token also expired). User redirected to login.	Both tokens cleared. nx_return_to stored.	<i>Redirect to /login. Banner: "Your session ended — log in to continue." After login: return to original page.</i>

12. Error Handling & Recovery Flows

Every error in NexaClip has a designed recovery path. Users should never reach a dead end. This section defines the response to all error categories.

12.1 — Error Categories & Responses

Error Category	User Sees	Recovery Action	Backend Behaviour
Form validation (client)	Inline red text below each invalid field on blur. Field border turns red.	Fix the field that is highlighted. Error clears when field becomes valid.	No API call made while form is invalid.
API error (4xx — user error)	Toast with specific, actionable message. Red border on relevant UI element.	Button to retry or correct the issue. Never just "Close".	HTTP 4xx with body: { message, errors? }.
API error (5xx — server error)	Toast: "Something went wrong on our end. We have been notified." Retry button.	Retry same action. If persists: "Contact support" link.	Error logged to CloudWatch. Sentry alert if configured.
Network timeout	Inline indicator: "Taking longer than usual..." after 5 seconds. "Try again" after 15 seconds.	Retry button. For long AI operations: "We will notify you when ready."	Timeout logged. Background retry for idempotent operations.
Offline state	Persistent amber banner at top: "You are offline." Updated to teal when reconnected.	Cached content remains viewable. Write operations queued for reconnection.	Angular service worker caches GET responses for offline viewing.
Auth expired	Silent token refresh attempted first. On failure: redirect to login.	Login page with return URL preserved.	Token refresh endpoint. Clear expired tokens.
Rate limit (API)	For AI: "Our AI is very busy right now. Your request is queued." For other: toast.	Wait and auto-retry for AI queue. Retry manually for other limits.	BullMQ queue with retry logic. 429 with Retry-After header.
404 Not Found	Custom 404 page. Illustration. "Page not found." Two CTAs.	Go to homepage. Go to previous page.	404 response. Angular catches unknown routes.

12.2 — Validation Flow — Signup Form

This exemplifies the validation pattern used consistently across all forms in the application.

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
1	User	Clicks into Email field. Types an invalid email: "notanemail".	No API call.	<i>Field: typing state — no error during typing.</i>
2	User	Clicks out of Email field (blur).	No API call.	<i>Client-side email format validation. If invalid: red border + "Please enter a</i>

#	Actor	What Happens On Screen	System / Backend Response	Browser / UI State
				<i>valid email address" below field. Field retains value.</i>
3	User	Types valid email. Clicks out.	POST /auth/check-email (debounced 400ms after blur).	<i>Validation loading: small spinner in field right. On response: if taken → red + "Email already registered. Log in →". If available: silent (no success indicator needed).</i>
4	User	Submits form with any remaining invalid fields.	No API call until all fields valid.	<i>All invalid fields highlight simultaneously. First invalid field scrolls into view. Focus moves to first invalid field.</i>
5	User	All fields valid. Submits form. Server returns 409 (race condition — username taken between check and submit).	POST /auth/register returns 409.	<i>Username field: red border + "This username was just taken. Try a different one." (specific error, not generic).</i>

12.3 — Graceful Degradation Scenarios

Scenario	Degraded Behaviour	Full Recovery
Claude API unavailable	Coach: "Coach is temporarily unavailable. Try again in a few minutes." Analytics reports: cached last available. Onboarding: skip to plan reveal with generic plan.	When Claude API recovers, queued jobs process normally. No data loss.
DALL-E unavailable	Image Studio: "Image generation is temporarily unavailable. Try again shortly." Stable Diffusion fallback if configured.	Automatic retry when DALL-E API recovers.
Redis unavailable	Rate limits disabled (allow all requests). Leaderboard served from PostgreSQL fallback (slower). Session data served from DB.	Redis reconnects automatically. State resumes.
S3 upload unavailable	Upload zone: "File storage is currently unavailable. Try again in a few minutes." No partial uploads.	Pre-signed URL generation retried. No files corrupted.
WebSocket disconnected	Real-time features (streaming Claude responses, live feed updates) fall back to polling. 5-second poll interval.	WebSocket reconnects automatically on network recovery. Streaming resumes.

13. Web-Specific UX Patterns

These patterns are specific to the desktop web experience and differ from typical mobile-first approaches.

13.1 — Keyboard Navigation

Keyboard Shortcut	Action
Ctrl/Cmd + K	Open command palette (global search + quick actions). Available on all app pages.
Ctrl/Cmd + Enter	Submit form or send message (multi-line textareas where Enter adds newline).
Escape	Close modal, dismiss sheet, cancel action.
Tab / Shift+Tab	Navigate between interactive elements (standard web pattern). Focus ring: 2px purple.
Arrow keys	Navigate within menus, select dropdowns, calendar date pickers.
Space	Activate focused button or toggle. Play/pause video when video player is focused.
/ (forward slash)	Focus the global search bar in TopBar when not in an input field.
G then F	Go to Feed (Gmail-style two-key navigation for power users).
G then C	Go to Create.
G then A	Go to Analytics.

13.2 — Hover States (Desktop Exclusive)

Element	Hover Behaviour
ContentCard in feed	Slight elevation lift (translateY -2px, shadow-sm → shadow-md, 150ms). Play button overlay appears on video thumbnails.
Follow button on cards	Changes from "Follow" to "Following ✓" text on hover IF already following (confirmation preview).
Navigation sidebar items	Background changes to navy-50. Left accent border appears (4px purple). Icon and text shift right 4px.
TopNav links	Purple underline slides in from left (150ms). Underline width matches text.
Table rows	Row background: white → gray-50. "View analytics" and "Share" inline actions fade in (opacity 0→1).
Stat cards	Shadow-sm → shadow-md. Subtle scale 1.0→1.01. Border color: gray-300 → purple-300.
Pricing card features list	Feature rows highlight with purple-50 bg on hover. Checkmark icon scales up slightly.
Coach context pills	Tooltip appears: "Click to update this context."

13.3 — Right-Click & Context Menus

Element	Right-click / Context Menu Behaviour
ContentCard thumbnail	Custom context menu: "Open in new tab", "Copy link", "Share to...", "Save image"
User avatar anywhere	Custom context menu: "View profile", "Follow/Unfollow"
Text in post caption	Browser native context menu (copy, search). No custom override.
SideNav items	No custom context menu — standard browser behaviour.

13.4 — Browser Tab Management

Scenario	Behaviour
Opening post in new tab	Ctrl+Click on ContentCard opens /feed/post/:id in a new tab as a full page (not SPA navigation). Complete app shell present in new tab.
Multiple NexaClip tabs	Each tab has independent JWT auth (all share the same localStorage token). Tab-to-tab state not synced — each tab has its own feed scroll position.
Tab title during loading	Shows "Loading... — NexaClip" until page data resolves, then updates to page-specific title.
Favicon badge	No unread notification count on favicon for MVP.

13.5 — Drag and Drop

Element	Drag & Drop Behaviour
Video upload zone on /create/clip	Full drop zone (entire dashed-border area). Drag-over: border turns solid purple, background purple-50, "Drop here" instruction text. Drop: file validation runs immediately.
Image upload on /profile/edit	Avatar circle becomes a drop zone on drag-over. Dashed border ring appears.
Content reordering (future)	Not in MVP scope. Timeline handles (trim editor) use drag but not HTML drag-and-drop — they use Pointer Events for cross-browser precision.

13.6 — Browser History & Navigation

Pattern	Implementation
Back button within app	Angular Router manages history stack. Back navigates to previous Angular route — not a page reload.
Back from post detail	Returning from /feed/post/:id via back button scrolls feed to the card that was clicked — scroll position preserved using Angular scroll restoration.
Back from upgrade	Returns to the page where upgrade was triggered (stored in sessionStorage as nx_return_to before leaving).
Page refresh in app	Angular app re-initialises. Auth check runs. Correct page re-renders. All API data re-fetched. No broken states from stale data.

Pattern	Implementation
Browser "Reopen closed tab"	Full page re-initialise. Same as page refresh.

14. Complete Screen & State Inventory

This table is the master reference for every screen in the NexaClip web app with its states and the Angular component responsible for it.

ID	Screen Name	Route	Component	States Required
W01	Homepage	/	HomePageComponent	Default, Scrolled (nav change), Hero demo: animating/complete
W04	Login	/login	LoginComponent	Default, Loading, Error (wrong creds), Error (locked), Success
W05	Signup	/signup	SignupComponent	Default, Email checking, Username checking, Password strength, Loading, Error (taken), Success
OB	Onboarding Chat	/onboarding	OnboardingComponent	Q1–Q5 states, Generating plan, Plan reveal, Skip
A01	Home Feed	/feed	FeedComponent	Loading (skeleton), Content (following), Content (discovery), Empty, Error, Pull-to-refresh
A02	Create Hub	/create	CreateHubComponent	Default, With context banner, Free limit banner, With recent items
A03	Image Studio	/create/image	ImageStudioComponent	Prompt empty, Prompt filled, Generating (loading), Result, Result with captions, Publishing, Success, Limit reached, Error
A04a	Clip Upload	/create/clip	ClipUploadComponent	Default, Drag-over, Uploading (progress), Complete, Error (file type), Error (file size), Error (network)
A04b	Clip Editor	/create/clip/:id/edit	ClipEditorComponent	Step 1 Trim: Loading frames, Timeline ready, Trimming, Preview. Step 2 Polish: Processing steps 1–3, Results ready. Step 3 Publish: Review, Publishing, Success
A05	Analytics	/analytics	AnalyticsDashboardComponent	Loading, Free (locked elements), Pro (full access), Empty (no posts), Report available (Monday), Error
A06	Creator Coach	/coach	CoachComponent	Gate screen (free), Loading history, Empty (first use), Conversation active,

ID	Screen Name	Route	Component	States Required
				Streaming response, Limit reached, Error
A07	Post Detail	/feed/post/:id	PostDetailComponent	Loading, Loaded (image), Loaded (video, muted), Comments loading, Comments loaded, Comments empty
A08	User Profile	/users/:id	UserProfileComponent	Loading, Loaded (not following), Loaded (following), Own profile (redirect), Empty (no posts), Private
A09	Own Profile	/profile	OwnProfileComponent	Loading, Loaded with posts, Loaded (no posts), Incomplete onboarding banner
A10	Edit Profile	/profile/edit	EditProfileComponent	Loading (prefill), Editing, Uploading avatar, Saving, Success, Error
A11	Content Library	/my-content	ContentLibraryComponent	Loading, All content, Filtered (clips/images/drafts/rejected), Empty filter, Bulk select mode, Deleting
A12	Upgrade Page	/upgrade	UpgradePageComponent	Monthly pricing, Annual pricing, Success (post-payment), Already Pro
A13	Settings	/settings	SettingsComponent	Account, Notifications, Billing (free), Billing (Pro — active), Billing (cancelling), Privacy, Danger Zone
ERR	404 Not Found	**	NotFoundComponent	Default only

15. Flow Summary & Designer Handoff Notes

15.1 — Journey Complexity Reference

Journey	Est. User Time	Screens Involved	Critical Success Metric
J1: First Visit & Registration	3–5 min	W01, W05, W04	Signup completion rate > 60% of homepage visitors who click CTA
J2: Onboarding Chat	90 seconds	OB (4 steps)	Completion rate > 85%. <10% skip.
J3: Image Creation	2–3 min	A02, A03	First image generated within 3 min of onboarding
J4: Clip Upload & Edit	5–8 min	A04a, A04b (3 steps)	Publish rate > 70% of users who complete upload
J5: Feed & Engagement	Open-ended (daily)	A01, A07	DAU/MAU ratio > 35%
J6: Profile Management	1–2 min	A08, A09, A10	Profile completion rate > 80% of registered users
J7: Analytics	2–5 min	A05	Pro: weekly report open rate > 70%. Free: upgrade conversion > 5%.
J8: Creator Coach	5–20 min	A06	Coach usage per Pro user: > 3 sessions/week
J9: Upgrade	2–4 min	Upgrade sheet, A12	Free-to-Pro conversion > 12% within 30 days of signup
J10: Settings	1–3 min	A13, A11	—

15.2 — Priority Order for Figma Prototype

The following prototypes should be built in Figma in this order to support early stakeholder demos and usability testing.

#	Prototype Name	Screens to Connect	Purpose
1	Core activation	W05 Signup → OB Chat → A02 Create Hub → A03 Image Studio → A01 Feed	The most-watched demo. Must feel fast and magical.
2	Upgrade moment	A03 (limit reached) → Upgrade sheet → A12 → Success → return to A03	Show investors the monetisation path.
3	Creator Coach	A06 full conversation (pre-scripted Claude responses)	Demonstrate AI value proposition.
4	Clip creation	A04a Upload → A04b Trim → A04b Polish → Feed post	Most technically impressive flow.

#	Prototype Name	Screens to Connect	Purpose
5	Analytics (Pro)	A05 full dashboard + weekly report + create from report → A03	Show the coaching intelligence loop.

Final Note to the Design & Architecture Team

This document covers every user journey in the NexaClip web application.

It is the definitive reference for how every screen connects, what every system response looks like, and how every error is handled.

Key principles to maintain throughout the design and build:

1. Web-first means desktop-first. Layouts are 12-column grid at 1280px+.

Every layout decision starts from the desktop and adapts down — never the reverse.

2. Every error has a recovery path. No dead ends. Every error state in this document includes a specific next action for the user.

3. Optimistic UI everywhere possible. Likes, follows, and comments update immediately without waiting for server confirmation. Only revert on failure.

4. The upgrade moment is never a wall. Free users see the product working before they see any paywall. The value is demonstrated before the price is shown.

5. Claude is a colleague, not a feature. Coach responses stream naturally.

There is no "AI mode" — Claude output appears in the same visual language as the rest of the product.

Confidential · For UI/UX Designer & Solution Architect · March 2026 · Desktop Web App — v1.0